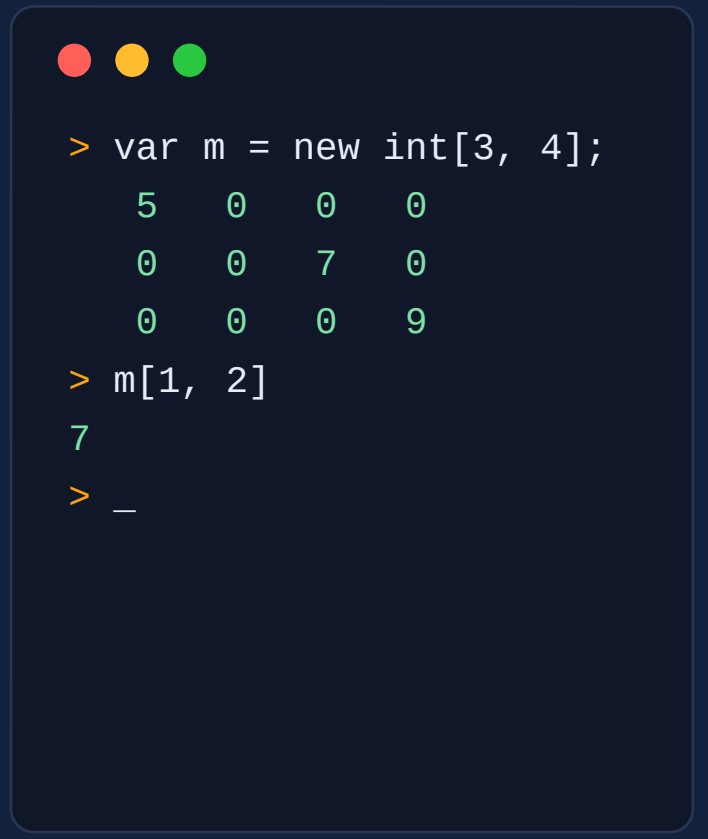
>_ MCMP113

Introduction to Basic Algorithms and Programming

Week 11

Multi-Dimensional Arrays (Matrices)

int[,] · nested loops · rows, columns, diagonals · matrix arithmetic · grids · jagged arrays



Asst. Prof. Hasan Oğuz, PhD
Istanbul Okan University · Vocational School
2026–2027 Fall Semester

By the End of Today You Can …

Declare and index

Create int[,] matrices, use m[r, c], GetLength(0) and GetLength(1).

Walk through a table

Visit every element with two nested for loops, row by row.

Rows, columns, diagonals

Compute row sums, column averages, maxima, the trace and the anti-diagonal.

Matrix arithmetic

Transpose, add, scale and multiply matrices, with size checks.

Model real grids

Grade books, cinema seats, tic-tac-toe and images are all matrices.

Jagged arrays

Use int[][] when the rows have different lengths (Pascal’s triangle).

From One Index to Two

Week 9 · one index

	0	1	2	3	4
0	18	21	19	**24**	22

```
double[] temps = new double[5];
temps[3] = 24;
```

One week of temperatures: **temps[3]** is the 4th day.

Week 11 · two indices

day →

week ↓

	0	1	2	3	4
0	18	21	19	24	22
1	17	16	20	**23**	25
2	22	19	18	24	21

```
double[,] temps = new double[3, 5];
temps[1, 3] = 23;
```



A matrix is an array with two indices: `m[row, column]`. Everything from Week 10 (sum, max, search, count) still works; we just add a second, nested loop.



PART 1

Rectangular Arrays

Declaring, indexing and walking through a table with int[,]

Tables Are Everywhere

Grade book

rows = students, columns = exams

Cinema hall

rows = seat rows, columns = seat numbers

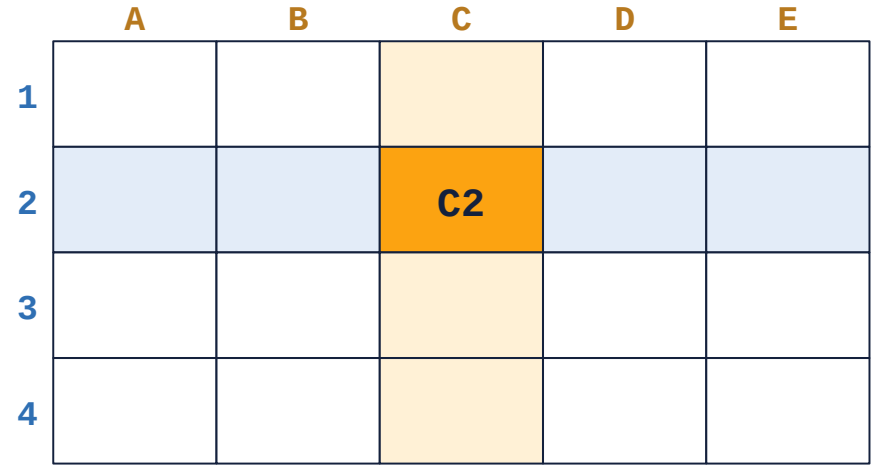
Board games

tic-tac-toe 3 × 3, chess 8 × 8

Images

rows × columns of pixel brightness values

A spreadsheet is a matrix



Cell C2 = row 2, column C.
Spreadsheets count rows from 1 and use letters for columns. In C# both indices start at 0, so the same cell is `m[1, 2]`.

>_ ANATOMY OF A 2-D ARRAY

int[,] m = new int[3, 4];

columns c = 0 … 3 →

	0	1	2	3
0	**5**	0	0	0
1	0	0	**7**	0
2	0	0	0	**9**

rows ↓

m[1, 2] = 7: row 1, column 2
m[0, 0] = 5: top-left corner
m[2, 3] = 9: bottom-right corner

```
int[,] m = new int[3, 4];

m[0, 0] = 5;
m[1, 2] = 7;
```

Piece	Meaning
`int[,]`	one comma = two dimensions (rows and columns)
`new int[3, 4]`	3 rows, 4 columns, 3 × 4 = 12 elements, all 0
`m[1, 2]`	row first, then column; both start at 0
a_{23} **(maths)**	counts from 1: the same element as m[1, 2]



Row first, then column — like a cinema ticket: row 12, seat 7.

Two Ways to Create a Matrix — and How Big Is It?

1 · Give the sizes

```
int[,] m = new int[3, 4];
```

Element type	Starts as
int, long	`0`
double	`0`
bool	`false`
char	`'\0'`
string	`null`

2 · Initializer

```
int[,] small = { { 1, 2, 3 }, { 4, 5, 6 } };
```

	0	1	2
0	1	2	3
1	**4**	5	6

2 inner { } = 2 rows
3 values each = 3 columns
`small[1, 0]` is 4

Size of m (3 × 4)

Expression	Value	Meaning
`m.GetLength(0)`	3	rows
`m.GetLength(1)`	4	columns
`m.Length`	12	all elements
`m.Rank`	2	dimensions



Length is not the number of rows! For loops over rows and columns use GetLength(0) and GetLength(1).

Set, Measure and Print a Matrix

```
int[,] m = new int[3, 4];

m[0, 0] = 5;
m[1, 2] = 7;
m[2, 3] = 9;

int rows = m.GetLength(0);
int cols = m.GetLength(1);
Console.WriteLine($"rows = {rows}, cols = {cols}");
Console.WriteLine($"Length = {m.Length}, Rank = {m.Rank}");

// row-major traversal: outer loop = rows, inner loop = columns
for (int r = 0; r < rows; r++)
{
    for (int c = 0; c < cols; c++)
    {
        Console.Write($"{m[r, c],4}");
    }
    Console.WriteLine();
}
```

code/11_01_MatrixBasics

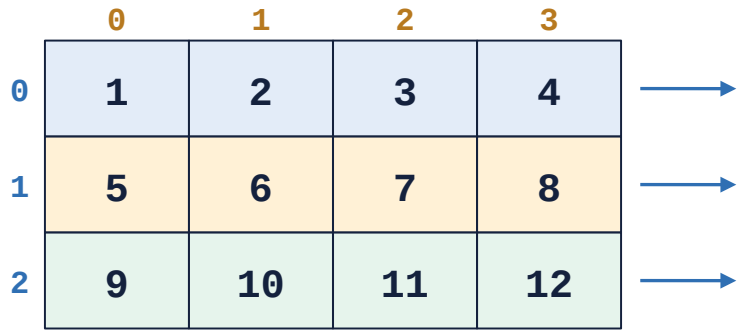
```
rows = 3, cols = 4
Length = 12, Rank = 2
   5   0   0   0
   0   0   7   0
   0   0   0   9
small[1, 0] = 4
foreach: 1 2 3 4 5 6
```

Reading the nested loop

- outer r: which row
- inner c: which column in that row
- WriteLine after the inner loop = new row

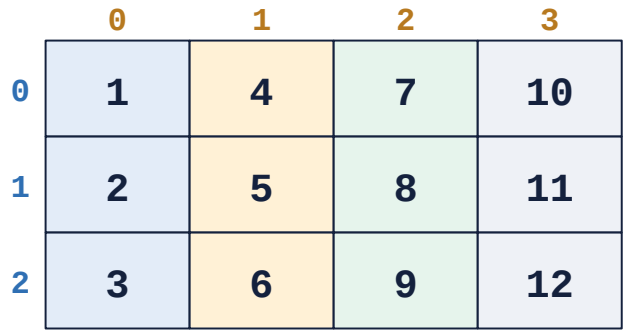
Row by Row or Column by Column?

Row-major: outer loop = rows




```
FOR r ← 0 TO rows − 1
  FOR c ← 0 TO cols − 1
    PRINT m[r, c]
  END FOR
END FOR
```

Column-major: outer loop = columns




```
FOR c ← 0 TO cols − 1
  FOR r ← 0 TO rows − 1
    PRINT m[r, c]
  END FOR
END FOR
```

The outer loop picks the line, the inner loop walks along it. The index inside m[r, c] is always row first — only the loop order changes.

Printing a Matrix So the Columns Line Up

```
int[,] m = { { 5, 120, 7 }, { 40, 3, 1000 } };
// inner statement of the usual nested loops:
```

1 · a space: ragged

```
Console.Write(m[r, c] + " ");
```

2 · width 5, right-aligned

```
Console.Write($"{m[r, c],5}");
```

3 · width −5, left-aligned

```
Console.Write($"{m[r, c],-5}|");
```

all three outputs: real runs

Format	Means
{x,5}	at least 5 characters, right-aligned
{x,-5}	at least 5 characters, left-aligned

Console

```
5 120 7
40 3 1000
```

Console

```
    5  120    7
   40    3 1000
```

Console

```
5    |120  |7    |
40   |3    |1000 |
```

Out of Range and Wrong Brackets

	0	1	2	3
0	5	0	0	0
1	0	0	7	0
2	0	0	0	9
3	?			

← m[3, 0]: there is no row 3

```
Console.WriteLine(m[3, 0]);   // m is 3 x 4
```

Console · run-time error (crash)

```
Unhandled exception.
System.IndexOutOfRangeException: Index was
outside the bounds of the array.
```

Wrong brackets: compile error

```
Console.WriteLine(m[1][2]);   // wrong
```

error CS0022: Wrong number of indices inside []; expected 2. Write **`m[1, 2]`**.

Swapped sizes: a silent logic error

```
for (int c = 0; c < m.GetLength(0); c++)
```

Columns need GetLength(1). On a square matrix the bug is invisible, so always test with a non-square one (3 × 4).



PART 2

Rows, Columns and Diagonals

Reading a matrix, sums and averages per row and per column, the two diagonals, the transpose

Reading a Matrix from the Keyboard

Pseudocode

```
START
  READ rows, cols
  FOR r ← 0 TO rows − 1
    FOR c ← 0 TO cols − 1
      READ a[r, c]
    END FOR
  END FOR
  …
```

Sizes first, then new int[rows, cols]: the size can come from variables.

```
Console.Write("Rows: ");
int rows = int.Parse(Console.ReadLine());
Console.Write("Columns: ");
int cols = int.Parse(Console.ReadLine());

int[,] a = new int[rows, cols];
for (int r = 0; r < rows; r++)
{
    for (int c = 0; c < cols; c++)
    {
        Console.Write($"a[{r},{c}] = ");
        a[r, c] = int.Parse(Console.ReadLine());
    }
}
```

Console

```
Rows: 2
Columns: 3
a[0,0] = 4
a[0,1] = 7
a[0,2] = 1
a[1,0] = 2
a[1,1] = 5
a[1,2] = 8
   4    7    1  |   12
   2    5    8  |   15
   6   12    9  |   27
```

Sum of Every Row, Sum of Every Column

	0	1	2	Σ
0	4	7	1	**12**
1	2	5	8	**15**
Σ	**6**	**12**	**9**	**27**

Row sum $R_r = \Sigma_c\, a[r, c]$

Column sum $K_c = \Sigma_r\, a[r, c]$

Total $\Sigma R = 12 + 15 = 27 = 6 + 12 + 9 = \Sigma K$

row sums: outer loop = rows

```
int total = 0;
for (int r = 0; r < rows; r++)
{
    int rowSum = 0;
    for (int c = 0; c < cols; c++)
    {
        Console.Write($"{a[r, c],5}");
        rowSum += a[r, c];
    }
    Console.WriteLine($"  | {rowSum,4}");
    total += rowSum;
}
```

column sums: outer loop = columns

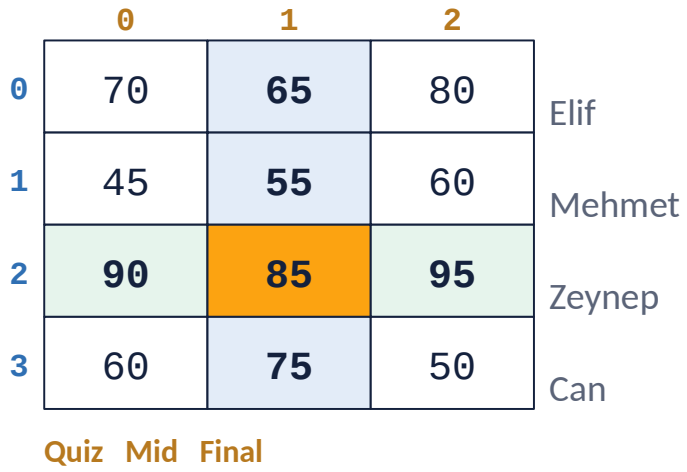
```
for (int c = 0; c < cols; c++)
{
    int colSum = 0;
    for (int r = 0; r < rows; r++)
    {
        colSum += a[r, c];
    }
    Console.Write($"{colSum,5}");
}
Console.WriteLine($"  | {total,4}");
```

A Grade Book: Students × Exams

```
string[] students = { "Elif", "Mehmet", "Zeynep", "Can" };
string[] exams = { "Quiz", "Midterm", "Final" };
double[,] grades =
{
    { 70, 65, 80 },
    { 45, 55, 60 },
    { 90, 85, 95 },
    { 60, 75, 50 },
};
int nStudents = grades.GetLength(0);
int nExams = grades.GetLength(1);
```

code/11_03_GradeBook

```
Student      Quiz  Midterm    Final  Average  Best
Elif           70       65       80    71.67    80
Mehmet         45       55       60    53.33    60
Zeynep         90       85       95    90.00    95
Can            60       75       50    61.67    75
Average     66.25    70.00    71.25
Top student: Zeynep (90.00)
```



a row (green) = one student: average and best grade

a column (blue) = one exam: exam average

grades[2, 1] = 85: Zeynep's midterm

Average and Best Grade of Every Row

```
for (int r = 0; r < nStudents; r++)
{
    double rowSum = 0;
    double rowMax = grades[r, 0];
    Console.Write($"{students[r],-8}");
    for (int c = 0; c < nExams; c++)
    {
        Console.Write($"{grades[r, c],9:F0}");
        rowSum += grades[r, c];
        if (grades[r, c] > rowMax)
        {
            rowMax = grades[r, c];
        }
    }
    double average = rowSum / nExams;
    Console.WriteLine($"{average,9:F2}{rowMax,6:F0}");
    if (average > topAverage)
    {
        topAverage = average;
        topStudent = r;
    }
}
```

Per row (inside the outer loop)

rowSum ← 0 and rowMax ← grades[r, 0]: a fresh start for every student.

Per element (inner loop)

print, add to rowSum, compare with rowMax.

After the row

average = rowSum / nExams; remember r if it is the best average so far.



Max starts at grades[r, 0], never at 0.

Main Diagonal, Anti-Diagonal, Triangles

Diagonals (n = 4)

	0	1	2	3
0	**2**	7	1	**8**
1	7	**3**	**5**	0
2	1	**5**	**4**	6
3	**8**	0	6	**9**

main: r = c
anti: r + c = n − 1

Triangles

	0	1	2	3
0	**2**	**7**	**1**	**8**
1	**7**	**3**	**5**	**0**
2	**1**	**5**	**4**	**6**
3	**8**	**0**	**6**	**9**

upper: c > r
lower: c < r

Part	Condition	Access
main diagonal	`r == c`	`a[i, i]`
anti-diagonal	`r + c == n - 1`	`a[i, n - 1 - i]`
upper triangle	`c > r`	—
lower triangle	`c < r`	—

trace = Σ a[i, i] = 2 + 3 + 4 + 9 = 18
anti = 8 + 5 + 5 + 8 = 26

One loop is enough for a diagonal: the column follows from the row.

Diagonal Sums and the Identity Matrix

both diagonal sums with one loop

```
int mainSum = 0;
int antiSum = 0;
for (int i = 0; i < n; i++)
{
    mainSum += a[i, i];
    antiSum += a[i, n - 1 - i];
}
Console.WriteLine($"Main diagonal sum (trace) = {mainSum}");
Console.WriteLine($"Anti-diagonal sum         = {antiSum}");
```

identity: 1 on the main diagonal, 0 elsewhere

```
int[,] identity = new int[n, n];     // all zeros
for (int i = 0; i < n; i++)
{
    identity[i, i] = 1;
}
Console.WriteLine($"Identity {n} x {n}:");
```

code/11_04_Diagonals

```
  [2]    7    1  (8)
    7  [3]  (5)    0
    1  (5)  [4]    6
  (8)    0    6  [9]
Main diagonal sum (trace) = 18
Anti-diagonal sum         = 26
Identity 4 x 4:
  1  0  0  0
  0  1  0  0
  0  0  1  0
  0  0  0  1
```

The Transpose: Rows Become Columns

A (2 × 3)

	0	1	2
0	1	2	3
1	4	5	6

A^T (3 × 2)

	0	1
0	1	4
1	2	5
2	3	6

Rule: `t[c, r] = a[r, c]` — sizes swap: 2 × 3 → 3 × 2

```
for (int r = 0; r < rows; r++)
{
    for (int c = 0; c < cols; c++)
    {
        t[c, r] = a[r, c];
    }
}
```

code/11_05_TransposeSymmetric

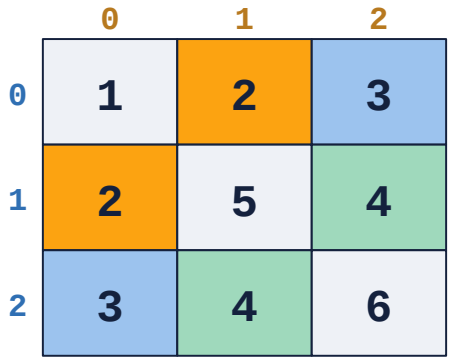
```
A (2 x 3):
   1   2   3
   4   5   6
Transpose (3 x 2):
   1   4
   2   5
   3   6
```

Classic bug

```
int[,] t = new int[rows, cols];   // wrong
```

For a 2 × 3 matrix t[2, 0] does not exist → IndexOutOfRangeException. Create t with the sizes swapped.

Is the Matrix Equal to Its Transpose?



Same colour = mirror pair.
S is symmetric: $S^T = S$

Assume true, then look for a counterexample

```
bool symmetric = true;
int checks = 0;
for (int r = 0; r < n; r++)
{
    for (int c = r + 1; c < n; c++)  // start right of the diagonal
    {
        checks++;
        if (s[r, c] != s[c, r])
        {
            symmetric = false;
        }
    }
}
Console.WriteLine($"S symmetric? {symmetric}  ({checks} comparisons)");
```

Console

```
S symmetric? True  (3 comparisons)
```

Why c = r + 1?

Only the upper triangle: n(n − 1)/2 = 3 pairs for n = 3, instead of 9 checks with c = 0.



PART 3

Matrix Arithmetic

Addition, scalar multiplication and the row-times-column rule

Addition and Scalar Multiplication

A

1	2	3
4	5	6

+

B

10	20	30
-1	-2	-3

=

A + B

11	22	33
3	3	3

3 ·

1	2	3
4	5	6

=

3	6	9
12	15	18

A + B: only if both have the same size
k · A: always defined

code/11_06_AddScale

```
Scalar k = 3
A + B =
   11   22   33
    3    3    3
```

continued

```
3 * A =
    3    6    9
   12   15   18
```

First check the sizes …

```
if (rows != b.GetLength(0) || cols != b.GetLength(1))
```

… then add element by element (else branch)

```
int[,] sum = new int[rows, cols];
for (int r = 0; r < rows; r++)
{
    for (int c = 0; c < cols; c++)
    {
        sum[r, c] = a[r, c] + b[r, c];
    }
}
```

Row Times Column

A (2 × 3)

	0	1	2
0	**1**	**2**	**3**
1	4	5	6

×

B (3 × 2)

	0	1
0	7	**8**
1	9	**10**
2	11	**12**

=

C (2 × 2)

	0	1
0	58	**64**
1	139	154

C[0, 1] = 1·8 + 2·10 + 3·12 = 8 + 20 + 36 = **64**

Formula $C[i, j] = \Sigma_k A[i, k] \cdot B[k, j], \quad k = 0 \ldots n - 1$

Sizes $(m \times n) \cdot (n \times p) \rightarrow m \times p$

The inner sizes must be equal (columns of A = rows of B); the outer sizes give the size of C. Otherwise A × B is not defined.

Inner loop for C[1, 1]

k	A[1,k]	B[k,1]	product	c[1,1]
–				0
0	4	8	32	32
1	5	10	50	82
2	6	12	72	**154**

? **Your turn:** compute C[1, 0] = row 1 of A × column 0 of B.

Three Nested Loops

```
if (n != b.GetLength(0))         // columns of A must = rows of B
{
    Console.WriteLine("A x B is not defined.");
}
else
{
    int[,] c = new int[m, p];        // result: m x p, starts at 0
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < p; j++)
        {
            for (int k = 0; k < n; k++)
            {
                // row i of A times column j of B
                c[i, j] += a[i, k] * b[k, j];
            }
        }
    }
    Console.WriteLine($"A ({m}x{n}) x B ({n}x{p}) = C ({m}x{p}):");
```

code/11_07_MatrixMultiply

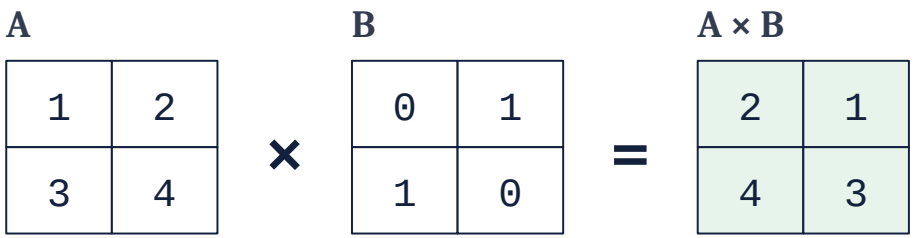
```
A (2x3) x B (3x2) = C (2x2):
    58    64
   139   154
Multiplications: 12
```

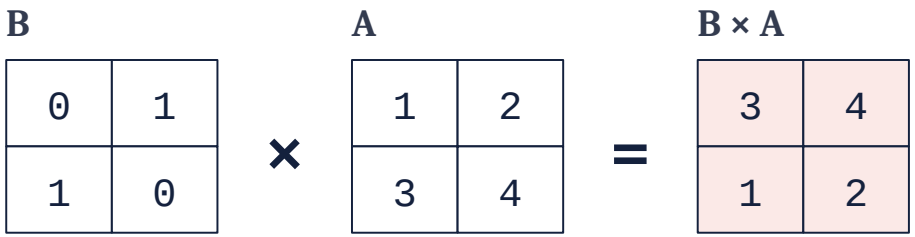
i **0 … m − 1** row of C

j **0 … p − 1** column of C

k **0 … n − 1** walks along row i of A and column j of B

A × B ≠ B × A, and the Identity





A × B swaps the columns of A, B × A swaps its rows.

Identity = the "1" of matrices: A × I = I × A = A. I has 1 on the main diagonal, 0 elsewhere (Example 4).

```
11_07 with these A and B · A × B
A (2x2) x B (2x2) = C (2x2):
     2     1
     4     3
```

```
initializers swapped · B × A
A (2x2) x B (2x2) = C (2x2):
     3     4
     1     2
```

Cost of n × n times n × n

n	multiplications n^3
2	8
10	1 000
100	1 000 000
1000	1 000 000 000



PART 4

Grids in Real Programs

Multiplication table, cinema seats, tic-tac-toe, images — and jagged arrays

The Multiplication Table as a Matrix

```
Console.Write("Size n (1-12): ");
int n = int.Parse(Console.ReadLine());

int[,] table = new int[n, n];
for (int r = 0; r < n; r++)
{
    for (int c = 0; c < n; c++)
    {
        table[r, c] = (r + 1) * (c + 1);
    }
}
```

```
if (n >= 8)
{
    Console.WriteLine($"7 x 8 = table[6, 7] = {table[6, 7]}");
}
```

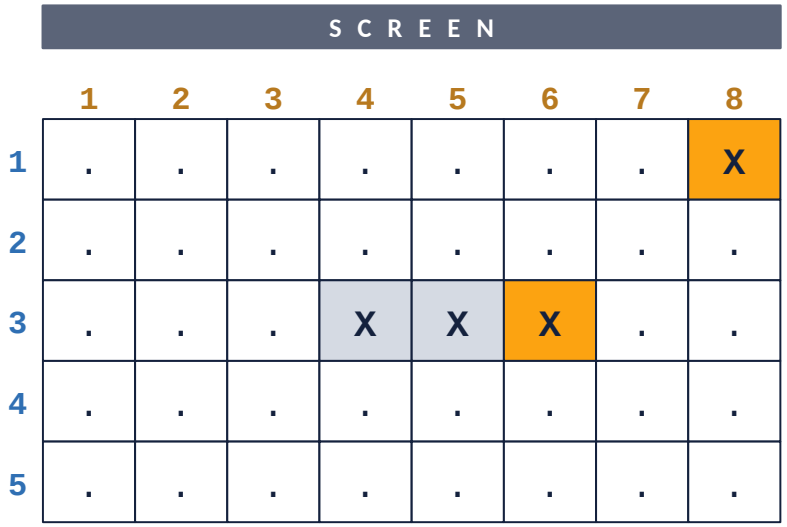


Index = number − 1: `7 × 8 → table[6, 7]`

code/11_08_MultiplicationTable

```
Size n (1-12): 9
   x|   1   2   3   4   5   6   7   8   9
   1|   1   2   3   4   5   6   7   8   9
   2|   2   4   6   8  10  12  14  16  18
   3|   3   6   9  12  15  18  21  24  27
   4|   4   8  12  16  20  24  28  32  36
   5|   5  10  15  20  25  30  35  40  45
   6|   6  12  18  24  30  36  42  48  54
   7|   7  14  21  28  35  42  49  56  63
   8|   8  16  24  32  40  48  56  64  72
   9|   9  18  27  36  45  54  63  72  81
7 x 8 = table[6, 7] = 56
```

Cinema Seats: a char Matrix



Seat numbers and **row numbers** as the user sees them (from 1). Row 3, seat 6 → **`seats[2, 5]`**.

■ booked in this run ■ sold before

> **Check the range first,** then the seat.

inside the do-while, after reading row and seat

```
if (row < 1 || row > rows || seat < 1 || seat > perRow)
{
    Console.WriteLine("  No such seat.");
}
else if (seats[row - 1, seat - 1] == 'X')
{
    Console.WriteLine("  Sorry, already sold.");
}
else
{
    seats[row - 1, seat - 1] = 'X';
    Console.WriteLine($"  Booked: row {row}, seat {seat}.");
}
```

Cinema Booking: the Console

code/11_09_CinemaBooking · part 1

```
Cinema: 5 rows x 8 seats
Row (1-5, 0 = finish): 3
Seat (1-8): 4
  Sorry, already sold.
Row (1-5, 0 = finish): 3
Seat (1-8): 6
  Booked: row 3, seat 6.
Row (1-5, 0 = finish): 3
Seat (1-8): 4
  Sorry, already sold.
Row (1-5, 0 = finish): 6
Seat (1-8): 1
  No such seat.
Row (1-5, 0 = finish): 1
Seat (1-8): 8
  Booked: row 1, seat 8.
Row (1-5, 0 = finish): 0
```

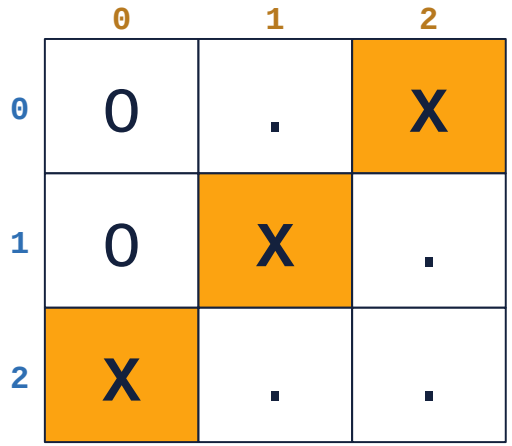
part 2 · the map

```
       S C R E E N
     1 2 3 4 5 6 7 8
Row1 . . . . . . . X
Row2 . . . . . . . .
Row3 . . . X X X . .
Row4 . . . . . . . .
Row5 . . . . . . . .
Free seats: 36 of 40
```

counting inside the printing loop

```
for (int c = 0; c < perRow; c++)
{
    Console.Write($" {seats[r, c]}");
    if (seats[r, c] == '.')
    {
        free++;
    }
}
```

Tic-Tac-Toe: Who Has Three in a Line?



8 lines to check

3 rows · 3 columns · 2 diagonals

main: r = c

anti: r + c = 2

anti-diagonal (the amber cells)

```
if (board[0, 2] == player
    && board[1, 1] == player
    && board[2, 0] == player)
{
    win = true;
}
```

rows and columns: one loop, i = 0, 1, 2

```
// win check: 3 rows, 3 columns, 2 diagonals
for (int i = 0; i < 3; i++)
{
    if (board[i, 0] == player
        && board[i, 1] == player
        && board[i, 2] == player)
    {
        win = true;
    }
    if (board[0, i] == player
        && board[1, i] == player
        && board[2, i] == player)
    {
        win = true;
    }
}
```

Tic-Tac-Toe: the Game Loop in Action

moves 1–3

```
X row: 2
X col: 2
  . . .
  . X .
  . . .
O row: 1
O col: 1
  O . .
  . X .
  . . .
X row: 1
X col: 3
  O . X
  . X .
  . . .
```

moves 4–5

```
O row: 2
O col: 2
Invalid move, try again.
O row: 2
O col: 1
  O . X
  O X .
  . . .
X row: 3
X col: 1
  O . X
  O X .
  X . .
X wins after 5 moves!
```

The game loop (pseudocode)

```
WHILE NOT win AND moves < 9
  READ row, col
  IF cell taken OR outside THEN
    PRINT "Invalid move"
  ELSE
    board[row, col] ← player
    moves ← moves + 1
    PRINT board
    check the 8 lines → win
    IF NOT win THEN
      switch X ↔ O
    END IF
  END IF
END WHILE
```

An Image Is a Matrix of Brightness Values

	0	1	2	3	4	5	6	7	8
0	0	6	9	6	0	6	9	6	0
1	6	9	9	9	6	9	9	9	6
2	9	9	9	9	9	9	9	9	9
3	3	9	9	9	9	9	9	9	3
4	0	3	9	9	9	9	9	3	0
5	0	0	3	9	9	9	3	0	0
6	0	0	0	3	9	3	0	0	0

0 = black … 9 = white (real photos: 0 … 255)

```
invert: inside the usual nested loops
image[r, c] = 9 - image[r, c];
```

part 1

```
Original:
  **@@**  **@@**
**@@@@@@**@@@@@@**
@@@@@@@@@@@@@@@@
--@@@@@@@@@@@@--
  --@@@@@@@@--
    --@@@@--
      --@@--
Average brightness: 5.76
```

part 2

```
Inverted:
@@--  --@@--  --@@
--      --      --

**              **
@@**          **@@
@@@@**      **@@@@
@@@@@@**  **@@@@@@
```

Jagged Arrays int[][]: Rows of Different Lengths

	0	1	2	3	4	5
0	1					
1	1	1				
2	1	2	1			
3	1	3	3	1		
4	1	**4**	**6**	4	1	
5	1	5	**10**	10	5	1

pascal[5][2] = 4 + 6 = **10** (sum of the two above)

```
int[][] pascal = new int[n][];          // n rows, none created yet
for (int r = 0; r < n; r++)
{
    pascal[r] = new int[r + 1];         // row r gets r + 1 elements
    pascal[r][0] = 1;                   // first and last element are 1
    pascal[r][r] = 1;
    for (int c = 1; c < r; c++)
    {
        // sum of the two numbers above
        pascal[r][c] = pascal[r - 1][c - 1] + pascal[r - 1][c];
    }
}
```

	int[,]	int[][]
element	`m[r, c]`	`j[r][c]`
shape	`rectangle`	`any length`
rows	`GetLength(0)`	`j.Length`
row length	`GetLength(1)`	`j[r].Length`

code/11_12_PascalJagged

```
Number of rows: 6
             1
           1   1
         1   2   1
       1   3   3   1
     1   4   6   4   1
   1   5  10  10   5   1
```

continued

```
Rows: 6
Last row length: 6
```

>_ COMMON MISTAKES

Six Matrix Bugs (All Real Messages)

`m[1][2]` on an `int[,]`

```
error CS0022: Wrong number of
indices inside []; expected 2
```

Fix: Write m[1, 2].

`{ { 1, 2 }, { 3 } }`

```
error CS0847: An array initializer
of length '2' is expected
```

Fix: Every row needs the same length (or use a jagged array).

`for (r = 0; r < m.Length; …)`

```
Length = 12 for 3 × 4 →
IndexOutOfRangeException
```

Fix: Rows: GetLength(0); columns: GetLength(1).

`c < m.GetLength(0)`

```
Works on 3 × 3, wrong on 3 × 4
```

Fix: Test with a non-square matrix.

`rowSum = 0` outside

```
Row 0: 3   Row 1: 10
(row 1 should be 7)
```

Fix: Reset the accumulator inside the outer loop.

`seats[row, seat]`

```
user types 1…5, indices are 0…4
```

Fix: Convert once: seats[row − 1, seat − 1].

>_ PREDICT THE OUTPUT · 3 MIN

What Does This Program Print?

```
int[,] q = { { 1, 2, 3 }, { 4, 5, 6 }, { 7, 8, 9 } };
int s = 0;
for (int r = 0; r < 3; r++)
{
    for (int c = 0; c < 3; c++)
    {
        if (c >= r)
        {
            s += q[r, c];
        }
    }
}
Console.WriteLine(s);
```

Choose one

A 45

B 26

C 15

D 34

Hint: shade the cells where c ≥ r.

Answer B: 26

	0	1	2
0	**1**	**2**	**3**
1	4	**5**	**6**
2	7	8	**9**

c ≥ r: the diagonal and everything to its right

$$1 + 2 + 3 + 5 + 6 + 9 = 26$$

Console

```
26
```

Condition	Cells	Output
`c >= r`	upper triangle + diagonal	26
`c == r`	main diagonal (trace)	15
`c <= r`	lower triangle + diagonal	34
`(no if)`	all nine	45

No if needed: start the inner loop at c = r.

>_ QUICK FIRE · 5 MIN

Three Short Questions

Q1 · Output?

```
int[,] m = { { 1, 2, 3 }, { 4, 5, 6 } };
for (int c = 0; c < 3; c++)
{
    for (int r = 0; r < 2; r++)
    {
        Console.Write($"{m[r, c]} ");
    }
    Console.WriteLine();
}
```

Q2 · Output?

```
int[,] g = new int[4, 6];
Console.WriteLine(
    $"{g.Length} {g.GetLength(0)}");
```

Q3 · What happens to a?

```
int[,] a = { { 1, 2 }, { 3, 4 } };
for (int r = 0; r < 2; r++)
{
    for (int c = 0; c < 2; c++)
    {
        int temp = a[r, c];
        a[r, c] = a[c, r];
        a[c, r] = temp;
    }
}
```



Q3 goal: transpose a in place. Does it work? If not, find the smallest change that fixes it.

Answers (Real Runs)

Q1 · columns become lines

Console

```
1 4
2 5
3 6
```

The transpose of m.

Q2 · all vs rows

Console

```
24 4
```

Length = 4 × 6 = 24 elements.

Q3 · every pair is swapped twice → a is unchanged

r	c	swap	a after the step
0	0	a[0,0] ↔ a[0,0]	1 2 / 3 4
0	1	a[0,1] ↔ a[1,0]	1 3 / 2 4
1	0	a[1,0] ↔ a[0,1]	**1 2 / 3 4**
1	1	a[1,1] ↔ a[1,1]	1 2 / 3 4

```
fix
for (int c = r + 1; c < 2; c++)
```

buggy run, then fixed run

```
1 2 / 3 4
1 3 / 2 4
```



Same idea as the symmetric check: visit only the upper triangle.

>_ IN-CLASS · PAIRS · 35 MIN

Your Turn

A Max of each row

Read a 3 × 4 matrix; print the largest value of every row and its column index.

★

B A – B

Subtract two matrices of the same size; print "not defined" if the sizes differ.

★

C Is it the identity?

Check whether a square matrix is the identity matrix.

★★

D Border sum

Sum of the elements on the border (first/last row, first/last column).

★★

E Study planner

Hours for 7 days × 4 weeks: total per day of week and per week.

★★

F Rotate 90°

Rotate an n × n matrix clockwise into a new matrix.

★★★

>_ HOMEWORK · DUE BEFORE WEEK 12

Practice at Home

Programs

- H1 · Example 2 with input validation: sizes 1–10, every element an integer (TryParse loops).
- H2 · Saddle point: find an element that is the smallest in its row and the largest in its column.
- H3 · Cinema: print the map after every booking and refuse bookings when the hall is full.
- H4 · Grade book: add a column “Final grade” = 30 % quiz + 30 % midterm + 40 % final.

On paper

- H5 · Multiply by hand and check with 11_07: [[2, 0], [1, 3]] × [[1, 4], [2, 5]].
- H6 · Count: how many times did today’s examples repeat the “print a matrix” loops? Next week we write them once, as a method.

Preview of Week 12: PrintMatrix(m) — write once, call everywhere.

>_ SUMMARY

Key Takeaways

01 int[,] m = new int[rows, cols]; m[r, c] — row first, both indices from 0.

02 GetLength(0) = rows, GetLength(1) = columns; Length = all elements.

03 Two nested loops visit every element; the outer loop picks the row (or column).

04 Reset row/column accumulators inside the outer loop; max starts at the first element.

05 Diagonals: a[i, i] and a[i, n − 1 − i]; transpose: t[c, r] = a[r, c].

06 A × B: C[i, j] = Σ A[i, k]·B[k, j], needs cols(A) = rows(B); A × B ≠ B × A.

07 Jagged int[][]: an array of rows, each with its own length.

NEXT WEEK

Functions (methods): definition, calling, parameters



MCMP113
Course materials